CMO² Strange Loop and GHL integration — full discussion record

Full record of the design discussion held on 15 Oct and 24-25 Oct 2026 (user timezone IST). Items marked (inferred) are my reading and need confirming; items marked (not verified) were not confirmed against primary documentation.

1. Decisions and conclusions at a glance

- Strange Loop is CMO²'s daily autonomous 5-stage marketing cycle; Stage 2 (Learning) and Stage 5 (Optimization) are still stubs.
- Sub-accounts are created in the GHL agency account by the owner, not from CMO². CMO² only learns each `locationId`.
- KPIs are calculated by CMO² itself from raw GHL inputs using standard calculations. GHL has no ready-made KPI or dashboard endpoint.
- Build the GHL layer on the official API. Use Snapshots (built by hand) for workflows, funnels and forms that clients inherit.
- The GHL CLI is a reference and admin tool, not a runtime dependency. Its internal-API builders stay out of the core product.
- The owner prefers the Snapshot-based approach for follow-up sequences: CMO² adds tags, GHL runs the sequence.

- The Strange Loop acts on GHL only through a capability registry, risk tiers, a guard layer, a per-tenant gateway and a credential vault.

2. What Strange Loop is in CMO²

Strange Loop OS is Phase 2 of the build. It is a 5-stage autonomous marketing engine, not a personal productivity system. Each cycle's output feeds the next cycle, which is the "loop". It runs daily, operated by the owner, with a partner covering when the owner is away. StayLockedIn.ai is the public storefront (GHL-hosted landing page); CMO² is the internal engine.

Stage	What it does	Status
1. Innovation	AI agents propose content and campaign ideas per client and industry (posts, blogs, funnels, avatar videos)	Infrastructure real
2. Learning	Reads results (leads, conversions, engagement) and works out what worked and why	Stub, no real AI reasoning yet
3. Documentation	Records what was done and learned; each day's context is saved in the database; Confluence and Notion are being scoped as outputs	Infrastructure real
4. Reporting	Turns results into reports and alerts	Infrastructure real

5. Optimization	Turns learnings into concrete changes that feed the next Innovation stage	Stub

- 17 AI strategist agents across 4 phases, with review gates RG-1 to RG-4.
- The Strange Loop tab shows the live cycle over WebSocket (STOMP).
- AiService is provider-agnostic. It defaults to a stub provider; the real path is Anthropic Claude.
- Stack: Java 17, Spring Boot 3.3, React 19, MySQL 8, Redis. Multi-tenancy is enforced at the ORM layer.

Note on terminology: "stub" means placeholder code. The structure and connections exist, but the function returns a dummy result and does not yet do the real work (in Hindi: अधूरे/placeholder code का हिस्सा, जिसमें असली logic बाद में भरा जाता है).

3. How to complete the Strange Loop

Suggested order, since each step depends on the one before:

1. Get real KPI data flowing: store KPI snapshots in MySQL per tenant, campaign and date, starting with GoHighLevel.

2. Build Stage 2 (Learning): a scheduled job compares this cycle's KPIs with the previous one in plain Java (deltas, trends, outliers), sends only a compact summary to `AiService`, and asks for structured JSON.
3. Build Stage 5 (Optimization): read Learning records, ask the AI for concrete actions, pass each through the review gates (human approval first, auto-approve low-risk later), execute approved actions through the connectors or GHL, and log them.
4. Close the loop: track each action as an experiment (hypothesis, action, expected result), measure the actual result next cycle and feed it back into Innovation.
5. Add guardrails: cap AI cost per tenant, keep actions idempotent, test with fixed KPI fixtures and the stub provider, and keep the tenant-leak tests running on new tables.

Principle: Java does the math, Claude does the thinking. Sending a small summary instead of raw data keeps cost and hallucination low.

Stage 2 sketch:

```
record KpiSnapshot(String campaignId, LocalDate date,
                   double spend, double clicks,
double leads) {}

record KpiDelta(String campaignId, String metric,
                double previous, double current,
double pctChange) {}

@Service
class LearningStage {
  private final KpiRepository kpis;
  private final AiService ai;
  private final LearningRepository learnings;

  List<KpiDelta> computeDeltas(Cycle prev, Cycle
curr) {
    List<KpiDelta> out = new ArrayList<>();
    for (var c : kpis.campaigns(curr)) {
      double p = kpis.sumLeads(c, prev);
      double n = kpis.sumLeads(c, curr);
      if (p > 0) out.add(new KpiDelta(c, "leads", p,
n, (n - p) / p * 100));
    }
    return out;
  }

  public LearningRecord run(Cycle prev, Cycle curr) {
    var notable = computeDeltas(prev, curr).stream()
        .filter(d -> Math.abs(d.pctChange()) > 15)
        .toList();
    String prompt = """
        You are a marketing analyst. Given these KPI
changes,
        return JSON only:
        {"worked":[], "failed":[], "hypotheses":[],
"confidence":0-1}
        Data: %s
```

```
        """.formatted(toJson(notable));
    AiLearning result = ai.generateJson(prompt,
AiLearning.class);
    return learnings.save(new
LearningRecord(curr.id(), notable, result));
  }
}
```

Flow: KPI snapshots (MySQL) → compare cycles (Java) → build compact summary → AiService (Claude) → Learning record → feeds Stage 5.

4. Getting KPIs from GHL

- Tokens come at two levels, Company (agency) and Location (sub-account). An agency token can request a location token through `/oauth/locationToken`. Listing an agency's sub-accounts needs an agency token; a location-scoped token gets a 403.
- Access tokens last about 24 hours. Agency-level tokens and OAuth are tied to Agency Pro, so confirm the plan.
- Rate limits (OAuth apps): 100 requests per 10 seconds and 200,000 per day, per app per resource (each location or company has its own budget; installing on more sub-accounts does not split it). Tracking headers: `X-RateLimit-Remaining`, `X-RateLimit-Daily-Remaining`, `X-RateLimit-Limit-Daily`, `X-RateLimit-Max`, `X-RateLimit-Interval-Milliseconds`. Read the headers instead of counting.

- There is no dashboard or KPI endpoint. Build KPIs from Contacts (lead volume), Opportunities (pipeline, win/loss), Payments (revenue), Conversations, and Calendars (bookings). Webhooks (50+ events) can replace polling for real-time events; use polling for a daily snapshot.
- Base URL is `https://services.leadconnectorhq.com` with a Bearer token and a `Version` header that varies by endpoint (the docs show v3, 2023-02-21 and 2021-07-28). A pasted set of cURL examples used `https://leadconnectorhq.com` with no path, which cannot work; check each endpoint's reference page.
- Open check: whether Social Planner exposes post performance numbers. Its section lists Post, Account, CSV, Statistics, OAuth, Comments, Category, Tag, Watermarks and Category Queue; the Statistics contents were not read.
- Official docs section list: Contacts, Conversations, Calendars, Opportunities, Payments (Orders, Fulfilments, Transactions, Subscriptions, Coupons), Campaigns, Email (Campaigns, Templates, Statistics), Funnels, Forms, Surveys, Social Planner, Snapshots, SaaS, Sub-Account, Users, Workflows, Webhooks and many more.

5. The GoHighLevel CLI project (attachment review)

What it is: a Python CLI (Click) with about 15 command groups, written by a third party (Lead Gen Jay), version 3.1.0. I read the README, SKILL.md, the API client, the subscription and lifecycle docs and the command list. I did not run it or its tests.

Two API layers:

Layer	Auth	Stability
Public API (`services.leadconnectorhq.com`)	Private Integration Token; agency token for SaaS calls	Official and stable
Internal API (`backend.leadconnectorhq.com`)	Firebase refresh token, which is the user's full GHL login	Unofficial, no SLA; the README says it "may run against GHL's ToS"

Command groups: contacts, opportunities, calendars, workflows, conversations, emails, smartlists, payments, saas, locations, forms, surveys, funnels, social, documents.

Internal-API features (all experimental, drafts only): workflow creation, form and survey creation, funnel page building, email campaign drafts, smart lists, contract templates. The project itself says to use these on your own agency account only and never on client sub-accounts, and to use Snapshots for client provisioning.

Reusable ideas from the CLI:

- Retry on 429/5xx, honour `Retry-After`, exponential backoff.
- Choose the API version by path prefix (2021-04-15 for conversations and calendars, `v3` for SaaS, 2021-07-28 for the rest).
- Never default the location ID (the CLI removed a hardcoded default after it silently targeted the wrong account).
- Dry-run by default for destructive actions, a protected location, and a fresh re-read after each write.
- GHL list endpoints are eventually consistent: do not confirm a write with an immediate read-back.
- Updating a workflow without `senderAddress` or canvas positions wipes them; send every field.
- Use `{{default contact.first_name "there"}}` for merge fields; a `||` fallback renders literally. Do not hand-code an unsubscribe footer; GHL adds one.
- 17 test files record these gotchas and can be ported as fixtures.

- Licence: the README says "Private / personal use" and the manifest tags it exclusive to its author's community. Confirm with the author before embedding it in a commercial product.

6. Official API versus CLI

Task	Official API	CLI
Contacts: create, update, search, tags	Yes	Yes
Opportunities and pipelines	Yes	Yes
Calendars: slots, booking, CRUD	Yes	Yes
Conversations: read, send	Yes	Yes
Payments: transactions, orders, subscriptions, invoices	Yes, plus coupons and fulfilments	Reads, plus create invoice

Sub-account read, search, tags, custom fields and values	Yes	Yes
Sub-account create, update	Yes (create cited in a third-party guide, not read in the official reference)	No
Sub-account pause, disable, delete; SaaS plans	Yes	Yes, with a dry-run safety wall and a billing audit
Social: accounts, posts	Yes, plus statistics, comments, CSV, categories, OAuth	Accounts and posts only
Email campaigns: create, edit draft, schedule	Yes (verified in the docs)	Internal API only; deliberately cannot send
Email statistics and templates	Yes (sections exist)	Not covered

Workflows: list, enrol, remove	Yes	Yes
Workflows: create, edit	Not confirmed; the CLI says no	Yes, as drafts, via the internal API
Forms and surveys: read submissions	Yes	Yes
Forms and surveys: create	Not confirmed	Yes, via the internal API
Funnels: list pages, count pages	Yes, read-only	Yes
Funnels: create or edit page content	No (matches the CLI's note)	Yes, via the internal API
Smart lists	Not found in the sections read	Create and list, via the internal API

Contract templates	Proposals section exists	Send, plus create and edit via the internal API
Snapshots	Read push history and create a share link; no push endpoint documented	None
Webhooks (50+ events)	Yes	None
OAuth and agency-to-location tokens	Yes	None (bearer tokens only)
Rate-limit headers	Yes	None; it only retries on errors

Official only (not touched by the CLI): webhooks, OAuth, social statistics and comments, email statistics, products, store, courses, blogs, Voice AI, Conversation AI, AI Agent Studio, knowledge base, users, custom objects and associations, files and media, trigger links, ad manager, LC Phone.

What the CLI adds beyond the API: internal-API writes, retry and version routing, lifecycle safety, the billing audit, local landing-page design tools (templates, lint, preview) that need no API, and written gotchas and tests.

Not verified: the Workflows reference page could not be opened, so whether the official API can create workflows rests on the CLI's note. Smart lists were not found in the official docs read. The pages read gave section names and short descriptions, not full parameters.

7. Worked example: 3-email follow-up for a new real estate lead

Goal: when a lead gets the tag `new-lead`, wait 5 minutes then send a welcome email, wait 1 day then send a second email, wait 2 days then send a third, and stop early if the lead gets the tag `booked-call`.

- Public API: can only list workflows and enrol a contact in one that already exists (`POST /contacts/{id}/workflow/{workflowId}`). It cannot create or edit workflows (per the CLI's notes).
- CLI: builds the workflow through the internal API using step helpers (tag, wait, email, goal) and saves it as a draft, authorised by the owner's full Firebase login.

- Risks of the internal route: one credential controls every client sub-account; GHL can change the routes without notice; wrong field names are accepted silently; refresh tokens expire or get revoked and need a manual re-login; possible terms-of-service exposure for the agency account.
- Chosen approach (Snapshots): build the workflow once by hand in the agency account, with a tag trigger on `new-lead` and a goal on `booked-call`. Save it in a Snapshot, one Snapshot per industry. Create each client sub-account from that Snapshot. CMO² then only adds the `new-lead` and `booked-call` tags through the official API.
- Prefer tag triggers over enrolling by workflow ID, because a copied workflow most likely gets a different ID in each sub-account (not verified).
- After creating a sub-account from the Snapshot, check that the workflow arrived and is published, send a test lead through, check the timing, and check that `booked-call` stops the sequence.
- Trade-off: CMO² cannot invent a brand new workflow for a client on the fly; anything new is built once by a human and shipped in a Snapshot. The same applies to forms, funnel pages, email campaigns and smart lists if the internal route is avoided.

Flow: build the sequence once (GHL screen) → save in a Snapshot → create client sub-account from it → CMO² tags the lead → GHL runs the sequence and stops on `booked-call`.

8. Can the CLI run inside the CMO² web app?

Technically yes, by starting the `ghl` command once per call with per-tenant environment variables. Not recommended:

1. Licence: "Private / personal use"; running it in a sold SaaS is commercial use.
2. Credentials are process-wide environment variables, so one process serving two tenants at once could mix tokens. The only safe pattern is one new process per call, with Python start-up cost each time.
3. No shared rate limiting: each process retries independently, so a daily run across all clients would hit 429s with no central per-tenant queue.
4. Fragile plumbing: a Python virtualenv, parsing output, manual token refresh, and the internal-API risks above.

Credentials also differ: the CLI expects a long-lived Private Integration Token per client, created by hand, while OAuth location tokens last about a day and would have to be refreshed and injected on every call.

Instead: port the public-API calls into a Java gateway; keep the CLI as the owner's admin tool (one profile per client for one-off checks, audits and support); port its test files as Java tests.

9. Credentials, capability registry and safety design

What clients should share:

Credential	Take it?	Why
GHL access	Not needed	The sub-account sits under the agency; the agency token or an OAuth app already reaches it
Social accounts	Let them connect inside GHL	GHL holds those tokens; CMO² posts through the GHL API and never sees the passwords
Other tools (ads, email, etc.)	OAuth through the owner's own app	Scoped and revocable by the client
Passwords, 2FA codes, a GHL Firebase refresh token	No, even if offered	A refresh token is the client's whole GHL login and only unlocks the unofficial internal API

Architecture: Strange Loop stages → capability registry (a fixed list of allowed actions) → guard layer (risk tier, approval, audit log) → GHL gateway (per-tenant queue, vault token) → client sub-account in GHL (official API only).

Risk tiers:

Tier	Examples	Rule
0 Read	contacts, opportunities, payments, conversations	Automatic
1 Reversible write	tags, custom values, workflow enrol/remove	Automatic and logged (the follow-up sequence lives here)
2 Client-facing	messages, social posts, bookings, invoices	Must pass the review gates RG-1 to RG-4
3 Destructive or money	deleting records, SaaS pause/disable/delete, subscription changes	Humans only; agents cannot call these

Three requirements:

1. A credential vault: encrypt each client's tokens, record which scopes they approved and when, support revocation, keep an audit log of every action. Request minimum scopes. The owner is UK-based and handles clients' leads' personal data, so look into data-processing terms and UK GDPR.
2. Prompt-injection protection: agents read lead messages and emails that may contain instructions. Agents must only pick from the fixed registry, never build free-form API calls, and treat message content as data. Tier 3 being closed to agents covers the worst case.
3. A daily run that survives token expiry: refresh OAuth location tokens, and mark a tenant unhealthy if a client revokes access.

10. Corrections made during the discussion

- Email campaigns: I first said creating campaigns needed the internal API. The official docs now list Create Email Campaign, Update Email Campaign (draft) and Schedule Campaign, so the CLI's note appears out of date. Whether Schedule actually sends is not stated and must be tested.
- Snapshots: I implied CMO² could apply a Snapshot through the API. The docs show no push endpoint (only share link and push-status reads), so applying a Snapshot remains a manual step.

- Gaps dropped by the owner: creating sub-accounts from CMO² and building KPI reporting inside the GHL connector are out of scope.

11. Open questions and next steps

Open questions:

1. Does one daily run process every client, or does each client have its own loop and memory? (inferred: per client per day)
2. Does Optimization act automatically on safe changes, or only propose changes for approval?
3. How do the 17 agents map onto the five stages? (not yet known)
4. Which GHL plan is the agency on? Agency Pro is needed for OAuth and agency API keys.
5. Does the official Workflows API allow creating or editing workflows? (not verified)

Possible next steps:

- Write the capability registry, with each CLI command mapped to a risk tier and the GHL scope it needs.
- Write the Java request client with per-tenant queuing, retries and version-by-path routing.
- Write a Java method that adds the `new-lead` or `booked-call` tag for a given tenant.

- Draw how the registry and risk tiers plug into each Strange Loop stage.

Sources

- GHL API documentation: https://marketplace.gohighlevel.com/docs/
- GHL help article on the API: https://help.gohighlevel.com/support/solutions/articles/48001060529-highlevel-api-documentation
- Rate limits: https://marketplace.gohighlevel.com/docs/other/rate-limits
- Email campaigns: https://marketplace.gohighlevel.com/docs/ghl/emails/campaigns
- Snapshots: https://marketplace.gohighlevel.com/docs/ghl/snapshots/snapshots
- Funnels: https://marketplace.gohighlevel.com/docs/ghl/funnels/funnel
- Social Planner: https://marketplace.gohighlevel.com/docs/ghl/social-planner/social-planner-api-v-3
- Sub-Account API: https://marketplace.gohighlevel.com/docs/ghl/locations/sub-account-formerly-location-api

- GoHighLevel API getting started (DEV Community): https://dev.to/autogencrm/getting-started-with-the-gohighlevel-api-auth-webhooks-and-common-gotchas-5fo3